Project: Preparing for Cloud Native

Due: 01/31/2026 11:59 PM
Extended Deadline: 02/02/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Your Very Own LLM Inference Server
- Task Overview
- Task

Infrastructure as Code
- Task Overview
- Task

k8s Primitives
- Task Overview
- Task

Wrap Up
- Conclusion

k8s Primitives (Task)

Task: Deploy Your LLM Inference Server to AKS

Introduction

In the previous task, you provisioned the cloud infrastructure required to host your LLM inference server using Terraform. Now, you'll move from infrastructure setup to **application deployment**. Your goal is to prepare your container image and define the Kubernetes configuration needed to run your Flask-based inference service on AKS.The instructions provided are less scaffolded to encourage independent problem-solving. However, if you find yourself stuck or spending too much time on a step, don't hesitate to ask for help in the discussion forum.

Environment Configuration

This task requires the kubectl tool. You need this tool to interact with your Kubernetes cluster. Be sure to install Kubectl before proceeding.

Task Overview

By the end of this task, you should have:

- Your container image stored in **Azure Container Registry (ACR)**
- Kubernetes **Deployment** and **Service** manifests created and applied to **Azure Kubernetes Service (AKS)**

A running Pod serving your Flask-based LLM inference endpoint

All Kubernetes configuration files should be organized under a new directory named `k8s` in your `flask-app` folder.

Sub-task 1: Push Your Container Image to ACR

Use the Azure CLI and Docker to tag and push your application image to your ACR instance. Make sure your image is accessible to the AKS cluster you created earlier.

[HINT]
You can simply tag the llmapp:latest you built in task1 with Azure's unique way of tagging container images. From there, push the image to the ACR.

Sub-task 2: Define the Deployment

Inside the `flask-app` directory, create the folder `k8s`, where you will add Kubernetes manifests that describe how your application should run on Kubernetes. Your Deployment should reference the image you pushed to ACR and configure any environment variables or ports required by your Flask app.

Consider:

Replica count
Container port
Resource requests/limits (optional but recommended)

(Hint: use `kubectl apply` and `kubectl get pods` to verify successful rollout.)

Note: Use this command to connect to your Kubernetes cluster before running the `kubectl` commands:

```
az aks get-credentials --resource-group
```

At the end of this sub-task, you should have created a `deployment.yaml` and applied the deployment

to the Kubernetes cluster.

Sample Deployment For A Generic App

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-deployment
  labels:
    app: generic-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: generic-app
  template:
    metadata:
    labels:
      app: generic-app
    spec:
      containers:
        - name: app-container
          image: your-container-image-u
          ports:
            - containerPort: 8080
```

Note: You cannot use the above configuration as-is. You need to make some modifications to make things work.

Sub-task 3: Define the Service

Next, create a Service manifest to expose your Deployment outside the cluster.

- Your service should be named, `llmapp-service`
- Decide which Service type best fits your setup (`ClusterIP`, `NodePort`, or `LoadBalancer`). In our case we want a service that is accessible from outside your cluster. That is, you should be able to use that endpoint in your browser and still get the expected output from your service.
- Ensure the Service routes traffic correctly to your Deployment.

At the end of this sub-task, you should have created a `service.yaml` and applied it to the Kubernetes cluster.

Sample Service For A Generic App

```
apiVersion: v1
kind: Service
metadata:
  name: app-service
  labels:
    app: generic-app
spec:
  selector:
    app: generic-app
  ports:
    - protocol: TCP
    port: 80
    targetPort: 8080
  type: ClusterIP
```

Note: You cannot use the above configuration as-is. You need to make some modifications to make things work.

Validate your Work

To validate your work, you should be able to use your service's external endpoint to access your inference server. You know this task is completed when after making a call to: `https://<external-service-ip-address>/api?message=hello`, you get the appropriate response.

Push your Code to GitHub

As best practice you should commit and push your code as frequently as possible.
Make sure you're in the root project directory, add, commit and push your changes to GitHub as you'll need this code for the next project.

Submit your Work

In your project root folder, run the following command to submit your work:

```
./submitter -t task3
```

(Compulsory) Knowledge Check On Troubleshooting & Debugging Strategies

Please fill out this survey on Troubleshooting & Debugging Cloud-Native Applications. This is not

graded. This is simply a quick knowledge check to help us understand where you are right now.

Destroy Your Resources

Once you're done with all tasks, you should go to the terraform directory and run the `terraform destroy` command. Run extra checks via the Azure portal to ensure that the resources are actually destroyed.

The Kubernetes cluster is an expensive resource. Forgetting to destroy will cause you to burn through your Azure credits quickly.

Please `do NOT` destroy the virtual machine you used as the development environment. You only have to shut it down. You will use this very VM in your next project.

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform